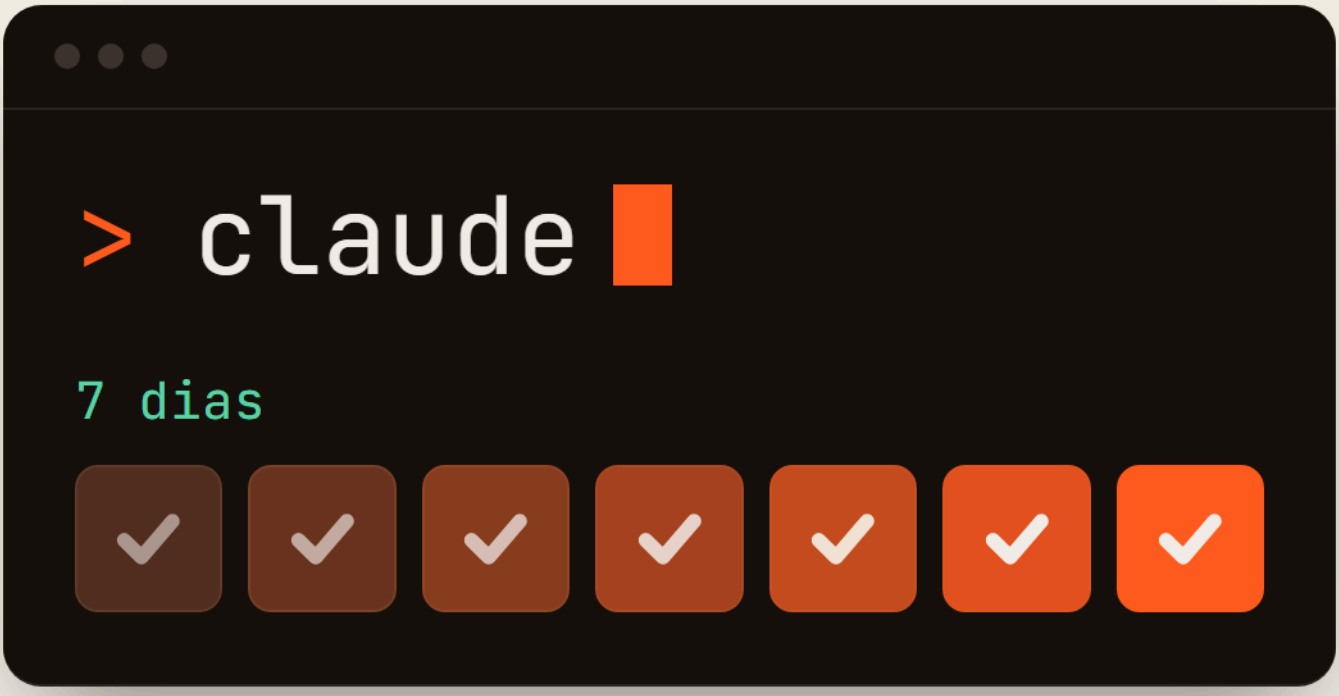
Sua 1ª semana no Claude Code

7 exercícios de até 15 minutos,
do primeiro comando à sua primeira skill



@clubedasskills

Grátis

Antes de começar

Em 7 dias, com 10 a 15 minutos de prática por dia, você instala o Claude Code, aprende a pedir com clareza e a controlar o que ele faz no seu computador. No fim da semana você tem uma página do seu produto feita com ele, um CLAUDE.md, uma skill e um subagente seus, e uma rotina que gasta menos limite.

O QUE VOCÊ PRECISA

- Um plano pago: Pro, Max, Ultra, Team, Enterprise ou Console (a API, com créditos pré-pagos). O plano grátis do claude.ai **não inclui** o Claude Code.
- Windows 10 (versão 1809 ou mais nova), macOS 13 ou mais novo, ou Ubuntu 20.04 ou mais novo.
- 4 GB de RAM e internet.
- O Brasil está na lista oficial de países atendidos.

Como usar este e-book

- Cada dia tem 2 páginas. Na 1ª você lê. Na 2ª você faz o exercício. O Dia 1 tem uma página a mais, só para erros de instalação.
- Tudo acontece numa pasta só, a `minha-semana`, com a página do seu produto ou serviço. Ela nasce no Dia 1 e cresce até o Dia 7.
- Nos pedidos prontos, troque o que está entre `[colchetes]` pelos seus dados antes de enviar.
- O Claude Code muda rápido, e muito tutorial antigo da internet já está errado. Onde algo mudou, você vai ver a palavra NOVIDADE.

Tudo aqui foi conferido na documentação oficial (code.claude.com/docs) em novembro de 2026. Se algo estiver diferente na sua tela, rode `/help`.

SUMÁRIO

DIA 1	Instalar e fazer o 1º pedido	3
DIA 2	Você no controle: permissões e planejamento	6
DIA 3	CLAUDE.md: ensine uma vez, vale sempre	8
DIA 4	Skills: o procedimento que você não repete mais	10
DIA 5	Plugins e MCP: ligue o Claude às suas ferramentas	12
DIA 6	Comandos, subagentes e tarefas agendadas	14
DIA 7	Economize limite e monte a sua rotina	16
→	Próximos passos	18

DIA 1

Instalar e fazer o 1º pedido

OBJETIVO terminar o dia com o Claude Code instalado e logado, e com o primeiro arquivo que ele criou numa pasta sua.

Abra o terminal

O terminal é a janela onde você digita comandos. É nele que o Claude Code roda.

- **Windows:** aperte `Win + X` e escolha Windows PowerShell (ou Terminal). Para colar, use `Ctrl + V` ou o clique direito.
- **Mac:** aperte `Cmd + Espaço`, digite `Terminal` e aperte Enter.

No Windows, confira onde você está: o PowerShell mostra `PS` no começo da linha. O CMD não mostra.

Instale

Copie a linha do seu sistema, cole no terminal e aperte Enter.

Windows (PowerShell):

```
irm https://claude.ai/install.ps1 | iex
```

Mac ou Linux:

```
curl -fsSL https://claude.ai/install.sh | bash
```

- O comando não mostra progresso. Deixe a janela aberta até aparecer "Claude Code successfully installed!" (Claude Code instalado com sucesso).
- No Windows, não precisa rodar como Administrador.
- Depois de instalado, ele se atualiza sozinho.

Para conferir, rode:

```
claude --version
```

Apareceu um número de versão seguido de `(Claude Code)`? Deu certo.

Crie a sua pasta e entre nela

```
mkdir minha-semana
cd minha-semana
claude
```

O `mkdir` cria a pasta. O `cd` entra nela. O `claude` abre o Claude Code ali dentro.

- Na 1ª vez, o login abre no navegador. Depois fica salvo.
- `/login` troca de conta. `/logout` sai da conta.

DICA

Nos próximos dias, abra o terminal, rode `cd minha-semana` e depois `claude -c`. O `-c` continua a última conversa desta pasta.

Faça o 1º pedido

- Escreva em português normal, como numa conversa.
- Ele lê os arquivos da pasta sozinho. Você não precisa colar nada.
- Se ele pedir permissão para criar um arquivo, escolha **Yes** (sim) e aperte Enter.
- Quer pular linha sem enviar? Digite `\` e aperte Enter.
- `/help` mostra os comandos.
- Para sair, digite `exit` ou aperte `Ctrl + D` duas vezes.

DEU ERRO?

`'irm' is not recognized` — você está no CMD. Abra o PowerShell ou use o comando do CMD:

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del
install.cmd
```

`The token '&&' is not a valid statement separator` — você está no PowerShell com o comando do CMD. Use o comando `irm` da página anterior.

`'claude' is not recognized` (Windows) — o Windows não achou o programa. Rode estas 2 linhas no PowerShell, feche a janela e abra outra:

```
$currentPath = [Environment]::GetEnvironmentVariable('PATH', 'User')
[Environment]::SetEnvironmentVariable('PATH',
"$currentPath;$env:USERPROFILE\.local\bin", 'User')
```

`command not found: claude` (Mac) — rode estas 2 linhas:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Erro de SSL/TLS (Windows 10 antigo) — rode esta linha e depois o comando de instalar de novo:

```
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
```

Outro erro? Rode `claude doctor` para um diagnóstico completo.

Prefere não usar o terminal? Tem o app Desktop, a extensão do VS Code, a do JetBrains e o site claude.ai/code.

Mão na massa

EXERCÍCIO

12 MIN

1. Instale e rode `claude --version`.
2. Rode `mkdir minha-semana`, depois `cd minha-semana`, depois `claude`. Faça o login.
3. Troque os colchetes e cole este pedido:

```
Cria um arquivo index.html com uma página simples para apresentar o que eu vendo:
[seu produto ou serviço], para [quem compra]. Coloca um título que diga o resultado
que a pessoa ganha, 3 benefícios curtos e um botão de contato. Quando terminar, me
diz como abrir essa página no navegador.
```

4. Abra a página do jeito que ele explicar.

Essa página vai com você até o Dia 7.

ERRO COMUM DO DIA

Pedir de forma vaga. "Melhora minha página" faz o Claude varrer tudo e escolher sozinho o que mudar. Diga o arquivo, o problema e o resultado que você espera.

Em vez de: "melhora minha página".

Peça: "no index.html, troca o título para [X] e deixa o botão maior".

ANOTAÇÕES

DIA 2

Você no controle: permissões e planejamento

OBJETIVO escolher quanta liberdade o Claude tem e fazer ele planejar antes de mexer.

Ele já começa no modo auto

NOVIDADE desde a versão 2.1.283, toda sessão nova começa no modo `auto`. Ele faz tudo sem perguntar, mas um classificador (um filtro automático) barra o que é arriscado: script baixado da internet (`curl | bash`), publicação em produção, dado sensível indo para fora. A doc avisa: o auto reduz as perguntas, mas não garante segurança.

Shift+Tab troca o modo

No Windows, se o `Shift+Tab` não funcionar, use `Alt+M`. A barra de status mostra o modo ligado.

Modo	Roda sem perguntar	Barra de status
Manual	só leitura	`⏸ manual mode on`
accept edits	edita arquivos e roda comandos comuns de arquivo na pasta	`⏵⏵ accept edits on`
plan	só pesquisa e propõe	`⏸ plan mode on`
auto	tudo, com o classificador	`⏵⏵ auto mode on`

- NOVIDADE o antigo modo `default` agora se chama Manual. Saindo do auto, o 1º toque vai para ele.
- Para abrir já num modo: `claude --permission-mode plan`.

Modo de planejamento

Ele pesquisa e propõe, sem editar. Comece o pedido com `/plan` ou use `Shift+Tab` até aparecer `⏸ plan mode on`. No fim, você escolhe: "Yes, and use auto mode" (sim, e usar o auto), "Yes, manually approve edits" (sim, eu aprovo cada edição) ou "No, keep planning" (não, continua planejando). `Ctrl+G` abre o plano no seu editor. `Shift+Tab` sai sem aprovar.

Freio e volta

- `Esc` para o Claude e mantém o que ele já fez. Aperte assim que ele sair do rumo. Num pedido de permissão, `Esc` recusa.
- `Esc` `Esc` com a caixa de texto vazia, ou `/rewind`, volta o código e a conversa a um ponto anterior.
- `/diff` mostra o que mudou.
- Enter enquanto ele trabalha põe sua mensagem na fila, sem interromper.

Permissão fixa

"Yes, and don't ask again" (sim, e não pergunte de novo) grava a regra em `.claude/settings.local.json`. O `/permissions` gerencia as regras allow (liberar), ask (perguntar) e deny (bloquear). A deny vale em todos os modos. O `bypassPermissions` libera tudo: só em container ou máquina virtual isolada.

Mão na massa

EXERCÍCIO

12 MIN

Na pasta `minha-semana`:

1. Troque os colchetes e cole:

```
/plan quero transformar o index.html numa página de vendas curta para [seu produto], com o preço [valor] e um botão de compra. Antes de mexer, me mostra o plano: quais seções, em que ordem e o que muda no arquivo.
```

2. Leia o plano e escolha "No, keep planning". Peça 1 ajuste.
3. Quando gostar, escolha "Yes, manually approve edits" e aprove cada edição.
4. Rode `/diff` e veja o que mudou.
5. Bônus de 1 minuto: mande um pedido qualquer e aperte `Esc` no meio.

ERRO COMUM DO DIA

Ir direto para a execução numa tarefa grande. Se a direção inicial estiver errada, refazer sai caro. Para mudança grande, comece com `/plan` ou `Shift+Tab` e aprove o caminho antes.

ANOTAÇÕES

DIA 3

CLAUDE.md: ensine uma vez, vale sempre

OBJETIVO criar o arquivo que o Claude lê no começo de toda sessão, curto e útil.

O que é

Um arquivo de instruções que o Claude lê no início de toda sessão. É contexto, não regra obrigatória ("context, not enforced configuration", diz a doc). Para bloquear algo de verdade, use deny (Dia 2) ou hook (Dia 6).

Onde fica

Os arquivos se somam:

- `~/.claude/CLAUDE.md`: você, em todos os projetos (o `~` é a sua pasta de usuário);
- `./CLAUDE.md`: este projeto;
- `./CLAUDE.local.md`: só você, neste projeto.

O /init

Ele analisa a pasta e cria um CLAUDE.md inicial. Se o arquivo já existe, sugere melhorias em vez de sobrescrever. Depois dele, diz a doc, o trabalho é mais apagar linhas.

O que pôr e o que tirar

Ponha o que ele não descobre lendo os arquivos: decisões, armadilhas, como conferir o trabalho, o que você já corrigiu 2 vezes.

Tire:

- a estrutura de pastas (teste da doc: ele acharia isso lendo o código, e iria olhar? Se sim, fora);
- procedimento longo, que vira skill (Dia 4);
- regra sem exceção, que vira hook ou deny.

Escreva instrução que dá para conferir: "preço sempre no formato R$ 97,00" em vez de "capriche nos preços". Fique abaixo de 200 linhas. Um comentário `<!-- nota -->` não gasta token (a unidade que mede quanto texto o Claude processa).

Conferir e editar

- `/memory` edita o arquivo e liga ou desliga a memória automática.
- `/context` lista os arquivos carregados em "Memory files".
- A memória automática vem ligada: ele anota sozinho o que aprende e avisa com "Saved 2 memories" (2 memórias salvas).
- "Lembra que..." vai para essa memória. "Adiciona isso no CLAUDE.md" vai para o arquivo.
- Depois de um `/compact` (Dia 7), o que foi dito só no chat pode se perder. O CLAUDE.md é relido.

Mão na massa

EXERCÍCIO

12 MIN

1. Na pasta `minha-semana`, rode `/init`.
2. Troque os colchetes e cole:

```
Edita o CLAUDE.md deste projeto e deixa no máximo 15 linhas, sem caixa alta. Apaga o que dá para descobrir lendo os arquivos e coloca: o que eu vendo ([produto]), para quem ([público]), o tom da página ([ex.: direto e informal]), o formato de preço que eu uso ([ex.: R$ 97,00]) e que você deve usar só os dados que eu passei e me perguntar quando faltar algum.
```

3. Agora cole:

```
Cria ou edita meu CLAUDE.md pessoal em ~/.claude/CLAUDE.md e acrescenta uma linha: responda sempre em português do Brasil, com frases curtas.
```

4. Saia com `exit` e abra de novo com `claude`. Rode `/context` e confira os 2 arquivos em "Memory files".

ERRO COMUM DO DIA

Gritar no CLAUDE.md. Segundo a doc, IMPORTANTE e CAIXA ALTA hoje mais atrapalham: ele aplica a regra demais. Regra que nunca pode falhar, como "nunca leia o .env" (o arquivo que costuma guardar senhas), é configuração. Use hook ou deny.

ANOTAÇÕES

DIA 4

Skills: o procedimento que você não repete mais

OBJETIVO criar a sua primeira skill e instalar as skills oficiais da Anthropic.

O que é

Uma skill é uma pasta com um arquivo `SKILL.md` de instruções. O Claude usa sozinho quando é relevante, ou você chama com `/nome-da-skill`. Segue o padrão aberto Agent Skills (agentskills.io), que funciona em várias ferramentas de IA.

É leve: só a descrição fica sempre carregada. Sinal de que você precisa de uma: você vive colando as mesmas instruções no chat.

Onde fica

- Pessoal: `~/.claude/skills/<nome>/SKILL.md`. Vale em todos os seus projetos.
- Projeto: `.claude/skills/<nome>/SKILL.md`.
- Plugin: você chama como `/plugin:skill`.

Editou a skill? Vale na hora, sem reiniciar. O `/skills` lista todas, e a tecla `t` ordena por tokens.

O essencial do arquivo

- `description`: o que a skill faz e quando usar. É assim que o Claude acha a skill.
- `disable-model-invocation: true`: só você pode chamar. Bom para publicar site ou salvar versão.
- O que você digita depois do nome chega em `$ARGUMENTS`.
- Mantenha o arquivo com menos de 500 linhas.

Já vêm prontas

`/code-review` revisa o que mudou atrás de bugs. `/simplify` revisa e limpa. `/loop` repete um pedido. `/doctor` faz um checkup.

Skills oficiais da Anthropic

Ficam no repositório `anthropics/skills`. Dentro da sessão, rode:

```
/plugin marketplace add anthropics/skills
/plugin install document-skills@anthropic-agent-skills
```

O `document-skills` trabalha com Excel, Word, PowerPoint e PDF. Também existe o `example-skills@anthropic-agent-skills`. Para usar, é só citar: "Use a skill de PDF para extrair os campos do formulário de [arquivo.pdf]".

- O repositório avisa: são para demonstração e estudo. Teste antes de usar em algo crítico.
- As skills `docx`, `pdf`, `pptx` e `xlsx` são "source-available, not open source" (código à vista, mas não aberto). Instale pelo comando oficial e não copie nem redistribua o conteúdo delas.

Mão na massa

EXERCÍCIO

15 MIN

1. Troque os colchetes e cole:

```
Cria uma skill pessoal em ~/.claude/skills/resposta-cliente/SKILL.md. Ela escreve respostas para mensagens de clientes sobre [seu produto]: tom [direto e simpático], no máximo 4 linhas, termina com uma pergunta e não inventa preço, prazo ou condição que eu não passei. A mensagem do cliente chega em $ARGUMENTS. Escreve uma description dizendo quando usar e me mostra o arquivo no fim.
```

2. Teste:

```
/resposta-cliente o cliente perguntou se dá para parcelar
```

3. Rode `/skills` e confira a skill na lista. Se ela não aparecer, saia com `exit` e abra de novo.
4. Para ajustar, peça "muda a skill resposta-cliente para..." Vale sem reiniciar.
5. Bônus: instale o `document-skills` com os 2 comandos da página anterior.

ERRO COMUM DO DIA

Deixar o CLAUDE.md gigante. Cada linha dele pesa em toda tarefa. Procedimento de vários passos vira skill, que só carrega quando é usada.

ANOTAÇÕES

DIA 5

Plugins e MCP: ligue o Claude às suas ferramentas

OBJETIVO instalar um plugin da loja oficial e entender como o Claude Code se liga a outros apps.

O que é um plugin

Um pacote que instala de uma vez skills, agentes, hooks e servidores MCP. Fez o bônus de ontem? O `document-skills` foi o seu primeiro.

- A loja oficial `claude-plugins-official` já vem adicionada, e os plugins dela se atualizam sozinhos.
- `/plugin` abre na aba Discover: digite para buscar. Na web, a vitrine fica em claude.com/marketplace.

Instalar e conferir

```
/plugin install commit-commands@claude-plugins-official
```

Escolha o escopo: só você (user), todos do repositório (project) ou só você neste repositório (local). Repositório é a pasta do projeto guardada com o Git. Se aparecer `Run /reload-plugins to apply.`, rode `/reload-plugins`.

Para conferir, digite `/` e procure `/commit-commands:commit`, ou rode `claude plugin list` no terminal. Também existem `claude plugin disable` e `claude plugin uninstall`. Leia a tela antes de instalar: um plugin pode rodar hooks e servidores MCP.

MCP

Padrão aberto que liga o Claude Code a ferramentas, bancos de dados e APIs. Use quando você vive copiando dados de outra ferramenta para o chat. No terminal, fora da sessão:

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

- `--scope` escolhe o alcance: `local` (padrão), `project` ou `user`.
- `claude mcp list` lista. `claude mcp remove notion` remove. Na sessão, `/mcp` mostra o status, faz o login e liga ou desliga.
- Prefira HTTP: SSE está obsoleto.

Conectores do claude.ai

O que você liga em claude.ai/customize/connectors aparece sozinho no Claude Code, se você entrou com a conta do claude.ai. Com chave de API, não carrega. No Team e no Enterprise, só o admin adiciona.

Cuidados

Só ligue servidor em que você confia: conteúdo de fora pode trazer prompt injection (texto escondido que tenta dar ordens ao Claude). Desligue em `/mcp` o que não usa, porque gasta limite. E, se a ferramenta tem programa de terminal próprio, prefira ele.

Mão na massa

EXERCÍCIO

12 MIN

1. Rode `/plugin` e fique na aba Discover. Abra 2 plugins e leia o que cada um instala.
2. Instale o `commit-commands@claude-plugins-official` no escopo local. Rode `/reload-plugins` se ele pedir. Digite `/` e ache `/commit-commands:commit`.
3. Cole:

```
Começa a salvar versões desta pasta com git: inicia o repositório e faz o commit das minhas mudanças com uma mensagem descritiva. Se o Git não estiver instalado, para e me explica como instalar.
```

4. Rode `/mcp` e veja o que já está conectado.
5. Usa Notion? Saia, rode o `claude mcp add` da página anterior, abra o Claude de novo e entre pelo `/mcp`.

DICA

Git é o programa que guarda versões dos seus arquivos. Cada versão salva é um commit. No Windows, o Git for Windows é opcional para o Claude Code, mas recomendado.

ERRO COMUM DO DIA

Achar que o "voltar" substitui o Git. O `/rewind` só desfaz o que o Claude editou com as ferramentas dele. Arquivo mexido por comando de terminal não volta. Por isso você fez o commit de hoje.

ANOTAÇÕES

DIA 6

Comandos, subagentes e tarefas agendadas

OBJETIVO passar trabalho para um assistente separado e deixar tarefas rodando sem você.

Comandos do dia a dia

Comando só funciona no começo da mensagem. Digite `/` para ver o menu.

- `/rename` dá nome à conversa. `/resume` volta a ela.
- `/btw` faz uma pergunta paralela que não entra no histórico.
- `/status` mostra versão, modelo e conta.
- `@` cita um arquivo. `!` roda um comando direto (`! git status`). `Alt+V` cola um print no Windows (`Ctrl+V` nos outros). `?` com a caixa vazia lista os atalhos.

Subagentes

Assistentes com contexto próprio: fazem a tarefa à parte e devolvem só o resumo. Explore, Plan e general-purpose já vêm prontos, e o Claude delega sozinho.

- Para criar o seu, peça ao Claude ou crie um arquivo em `.claude/agents/` (projeto) ou `~/.claude/agents/` (pessoal). Chame em linguagem normal ou com `@`.
- NOVIDADE o `/agents` não abre mais tela.
- Cada subagente conta no mesmo limite. Para tarefa simples, `model: haiku` no arquivo. Delegar não deixa a resposta mais inteligente.
- `Ctrl+B` manda para segundo plano. `/tasks` mostra o que está rodando.

/loop e lembretes

Só com a sessão aberta. `/loop 5m [pedido]` repete a cada 5 minutos (unidades `s`, `m`, `h` e `d`), expira em 7 dias e para quando você fecha o terminal. Lembrete único: "me lembra às 15h de [tarefa]". Para listar ou cancelar, é só pedir.

Tarefas agendadas na nuvem

NOVIDADE as antigas "routines" agora são tarefas agendadas. O comando é `/schedule`, e `/routines` virou apelido. Elas rodam na nuvem da Anthropic, com o PC desligado, a cada 1 hora no mínimo, numa cópia nova do repositório (sem seus arquivos locais). Estão em pré-visualização, nos planos Pro, Max, Ultra, Team e Enterprise, e precisam de login no claude.ai: chave de API não serve. Crie em claude.ai/code/routines ou com `/schedule`.

Hooks

Comando que roda sempre num momento fixo (antes de editar, ao terminar...), sem depender do Claude decidir. É aqui que mora a regra "nunca". Fica no `settings.json`, e o `/hooks` mostra os configurados. Peça: "escreve um hook que bloqueia edição na pasta [X]".

Mão na massa

EXERCÍCIO

15 MIN

1. Cole:

```
Cria um subagente em .claude/agents/revisor-de-pagina.md, só com ferramentas de leitura (Read, Grep, Glob). Ele revisa páginas de venda: confere se o título diz o resultado, se o preço está claro, se o botão diz o que acontece ao clicar e se tem erro de português, e devolve só a lista de problemas, sem reescrever a página. Depois usa esse subagente para revisar o index.html.
```

2. Com a lista na tela, peça:

```
Corrige os 3 problemas mais importantes da lista.
```

3. Rode `/diff`.
4. Bônus de 1 minuto (deixe a sessão aberta):

```
me lembra às [hora] de revisar a página no celular
```

ERRO COMUM DO DIA

Não dar ao Claude um jeito de conferir o trabalho. Sem teste ou print para comparar, "parece pronto" é tudo o que ele tem, e você vira o testador. Peça a prova (o comando que ele rodou e a saída) ou cole um print com `Alt+V`.

ANOTAÇÕES

DIA 7

Economize limite e monte a sua rotina

OBJETIVO entender o que gasta o seu limite e sair com uma rotina diária que rende mais.

Por que o limite acaba

Toda mensagem reenvia a conversa inteira. Quanto maior o contexto, maior o custo.

- `/context` mostra uma grade do que ocupa espaço.
- `/usage` mostra custo, limites e estatísticas. NOVIDADE `/cost` e `/stats` viraram apelidos dele.
- Nos planos pagos, o `/usage` mostra quanto gastou cada skill, subagente, plugin e MCP, com recomendações. A tecla `f` aplica as que ele consegue (versão 2.1.305+).

/clear e /compact

- `/clear` entre tarefas sem relação. Não custa nada. Antes, use `/rename`, ou `/clear [nome]`, para achar a conversa depois no `/resume`.
- Duas correções não deram certo? `/clear` e um pedido novo e melhor funcionam melhor quase sempre.
- `/compact [foco]` resume a conversa para você continuar nela. Mas, numa sessão longa, o próprio `/compact` é uma requisição grande.

Modelo e esforço

- Padrão: Opus 5.5, com esforço `medium`.
- `/model` troca o modelo. Sonnet custa menos na rotina. `Alt+P` (no Mac, `Option+P`) troca sem apagar o que você escreveu.
- `/effort low` serve para ida e volta rápida. `max` só para problema difícil: em tarefa comum, ele pensa demais.
- A palavra `ultrathink` pede raciocínio mais fundo só naquele pedido. "Pensa bastante" não é comando.
- O limite de sessão e o semanal valem para todos os modelos juntos: trocar não resolve. Só o aviso "You've hit your Opus limit" (limite do Opus) se resolve com `/model`.

O que gasta sem você ver

`/loop`, tarefas agendadas e subagentes continuam mandando requisições. MCP ligado sem uso também gasta. E pausa longa custa: o cache (a parte da conversa já processada) dura cerca de 1 hora na assinatura e 5 minutos com chave de API. Depois disso, a 1ª mensagem reprocessa tudo.

Sua rotina

1. Abra com `claude -c` ou `/resume`.
2. Peça algo específico, citando o arquivo com `@`.
3. Tarefa grande começa com `/plan`.
4. Mudou de assunto: `/clear`.
5. 2 correções falharam: `/clear` e um pedido melhor.
6. No fim do dia: `/usage`.

De vez em quando, rode `/doctor`: ele sugere cortes no CLAUDE.md.

Mão na massa

EXERCÍCIO

12 MIN

Raio-x do seu limite:

1. Rode `/context` e anote o que mais ocupa.
2. Rode `/usage`.
3. Cole:

```
Analisa o CLAUDE.md deste projeto, o meu CLAUDE.md pessoal e as skills que eu criei esta semana. Me diz o que dá para cortar ou mover para uma skill para gastar menos contexto. Não muda nada ainda, só me mostra a lista.
```

4. Aplique o que fizer sentido e termine com `/clear`.

ERRO COMUM DO DIA

Misturar tarefas sem relação na mesma sessão. A conversa velha vai junto em toda mensagem e gasta limite. Use `/clear` entre tarefas: não custa nada.

ANOTAÇÕES

Próximos passos

Marque o que você já faz:

- [ ] O Claude Code está instalado e o `claude --version` responde.
- [ ] Sei trocar de modo com `Shift+Tab` e uso `/plan` antes de tarefa grande.
- [ ] Meu CLAUDE.md tem menos de 200 linhas e nada em caixa alta.
- [ ] Criei 1 skill minha e chamei com `/nome-da-skill`.
- [ ] Instalei 1 plugin pelo `/plugin` e sei ver o que está ligado no `/mcp`.
- [ ] Criei 1 subagente e sei usar o `/loop` ou um lembrete.
- [ ] Uso `/clear` quando troco de tarefa e confiro o `/usage` no fim do dia.

Quer ir mais rápido?

Quer pular a parte de escrever skills do zero? No @clubedasskills tem pacotes de skills prontas para o Claude Code, por apenas R$39,90. O link está na bio. E segue a gente para mais dicas de Claude Code.

@clubedasskills

Conferido na documentação oficial do Claude Code em novembro de 2026.